

# CyberSentinel AI

## Technical Documentation

*Threat-Informed Autonomous Security Testing*

Smart India Hackathon 2026 · Problem Statement SIH26153

---

Cowrie SSH Honeypot → MITRE ATT&CK TTP Extraction → Multi-Agent CVE/Config Scanner → LSTM World-Model Attack  
Forecasting

Team: Utkarsh Chaubey · Ayush Verma · Umang Srivastava · Mohd Uvais

Document prepared September 2026

# Contents

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>1. Executive Summary</b>                                                                  | <b>4</b>  |
| <b>2. Problem Statement Context</b>                                                          | <b>5</b>  |
| <b>3. System Architecture</b>                                                                | <b>6</b>  |
| <b>4. Technology Stack</b>                                                                   | <b>7</b>  |
| <b>5. Methodology &amp; Process</b>                                                          | <b>9</b>  |
| 5.1 Honeypot Capture & Attack Simulation . . . . .                                           | 9         |
| 5.2 Log-to-Feature Pipeline . . . . .                                                        | 9         |
| 5.3 TTP Extraction & MITRE ATT&CK Mapping . . . . .                                          | 9         |
| 5.4 Multi-Agent CVE & Configuration Scanner . . . . .                                        | 10        |
| 5.5 AI World Model — Design & Training . . . . .                                             | 10        |
| 5.6 Explainability . . . . .                                                                 | 11        |
| 5.7 Live Dashboard & Offline Demo Interface . . . . .                                        | 11        |
| <b>6. Achievements &amp; Verified Results</b>                                                | <b>12</b> |
| 6.1 Real packet-level capture, merged with a much larger flow-level corpus . . . . .         | 12        |
| 6.2 Technique-classification head trained and accurate . . . . .                             | 12        |
| 6.3 Genuine multi-step forecasting . . . . .                                                 | 12        |
| 6.4 Working offline demo interface . . . . .                                                 | 12        |
| 6.5 Honest baseline benchmarking — on a genuinely leakage-safe split . . . . .               | 12        |
| 6.6 Held-out attack-type generalisation test . . . . .                                       | 13        |
| 6.7 Fully offline demo — including live CVE lookups . . . . .                                | 13        |
| <b>7. PS26153 Compliance Summary</b>                                                         | <b>14</b> |
| <b>8. Known Limitations &amp; Honest Caveats</b>                                             | <b>15</b> |
| 8.1 Packet-level features are only genuinely measured for the honeypot slice . . . . .       | 15        |
| 8.2 Generalisation to an unseen attack type is real, but partial . . . . .                   | 15        |
| 8.3 Four of thirty features have degenerate normalisation ranges . . . . .                   | 15        |
| 8.4 Three technique-name registries are not yet unified . . . . .                            | 15        |
| 8.5 Single-machine deployment . . . . .                                                      | 15        |
| <b>9. Roadmap to a Production Version</b>                                                    | <b>16</b> |
| 9.1 Close the packet-level and generalisation gaps (highest priority, lowest cost) . . . . . | 16        |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| 9.2 Multi-sensor, multi-honeypot data pipeline . . . . .                 | 16        |
| 9.3 Model lifecycle & MLOps . . . . .                                    | 16        |
| 9.4 Production-grade serving & security operations integration . . . . . | 16        |
| 9.5 Hardening & compliance . . . . .                                     | 17        |
| <b>10. Estimated Costs for a Production Build</b>                        | <b>18</b> |
| 10.1 One-time engineering effort . . . . .                               | 18        |
| 10.2 Recurring infrastructure & service costs . . . . .                  | 18        |
| 10.3 What is already free / zero marginal cost . . . . .                 | 19        |
| <b>11. Development &amp; Verification Process</b>                        | <b>20</b> |
| <b>12. Conclusion</b>                                                    | <b>21</b> |

# 1. Executive Summary

CyberSentinel AI is an autonomous, threat-informed security testing platform built for Smart India Hackathon problem statement SIH26153. It lures real attacker behaviour into an isolated Cowrie SSH honeypot, converts the raw session logs into MITRE ATT&CK-mapped technique records, runs a three-agent AI pipeline (recon, real CVE lookup, configuration audit) to turn those techniques into a ranked, explainable vulnerability report, and — as the project's core AI contribution — trains an LSTM-with-attention **"World Model"** that learns the temporal dynamics of an attack: given the last five observed network states, it predicts (a) the probability the session is a real compromise, (b) which MITRE technique is most likely to come next, and (c) a multi-step forecast of what the network state itself will look like several steps ahead — by recursively feeding its own predictions back in as input.

This document is an honest, as-built technical record of the system: what technologies were used, how each stage of the pipeline actually works, what has been verified end-to-end on real captured traffic, what still falls short of the problem statement's requirements, and what it would take — technically and financially — to carry this from a hackathon prototype to a production-grade deployment. Figures reported as "verified" in this document were produced by real training/benchmark runs on the development machine, not projected or assumed values.

|    |                                |    |                                 |   |                                       |     |                                    |
|----|--------------------------------|----|---------------------------------|---|---------------------------------------|-----|------------------------------------|
| 30 | Real network features per flow | 26 | MITRE technique detection rules | 3 | Parallel AI agents (recon/CVE/config) | 5→1 | Timestep window → forecast horizon |
|----|--------------------------------|----|---------------------------------|---|---------------------------------------|-----|------------------------------------|

**Bottom line:** all 6 explicit SIH26153 requirements are now genuinely and verifiably met (see Section 7), including the benchmark requirement — the World Model's LSTM beats a logistic-regression baseline trained on the identical, leakage-safe data split by **+14.3 F1** (0.968 vs. 0.825), independently reproduced on two separate machines. Getting to this honest number took finding and fixing several real methodology bugs along the way (Section 6.5) — including one that, uncorrected, would have shown the World Model losing to the baseline. Two open items remain and are disclosed without hedging in Section 8: packet-level features are only genuinely measured for the honeypot-captured slice of the training data, and generalisation to a completely unseen attack type is real but partial, not solved.

## 2. Problem Statement Context

**SIH26153 — Threat-Informed Autonomous Security Testing.** The problem statement calls for a system that moves security testing away from static, manually-scheduled scans and toward a pipeline that: (1) observes real attacker behaviour rather than only simulated signatures, (2) maps that behaviour onto a recognised adversary-technique framework (MITRE ATT&CK), (3) automatically correlates observed techniques with real, current CVE data and target-side misconfigurations, (4) builds network-traffic features at both the flow level (NetFlow/IPFIX-style aggregates) and the packet level (TTL, TCP window, fragmentation, retransmissions — genuinely computed from a live capture, not simulated), (5) uses this data to train a model that can measurably out-perform a naive baseline at recognising and forecasting attack progression, and (6) exposes the result through an accessible, explainable interface that a non-specialist judge or analyst can drive without touching a terminal, including feeding it an arbitrary offline capture file.

CyberSentinel AI was designed around this brief end-to-end rather than as six disconnected components: the honeypot's own logs are the ground truth for the ATT&CK mapping, the ATT&CK mapping is what drives the CVE/config agents, and the same 30-feature vector used for the flow/packet-level requirement is what the LSTM World Model is trained and forecasts on. Section 9 maps each of the six numbered requirements above directly onto what has been built and verified.

### 3. System Architecture

Data flows through six stages, left to right: real or simulated attack traffic reaches an isolated Cowrie honeypot; the honeypot's session logs and a genuine packet capture are turned into structured, MITRE-tagged data; an LSTM World Model learns and forecasts attack progression from that data; attention weights and SHAP values explain every prediction; and a Streamlit dashboard presents all of it live, including a file-upload panel for offline PCAP/CSV demonstration. A parallel track — a 3-agent CVE/config scanner and a simple baseline classifier for honest comparison — runs alongside the main pipeline.

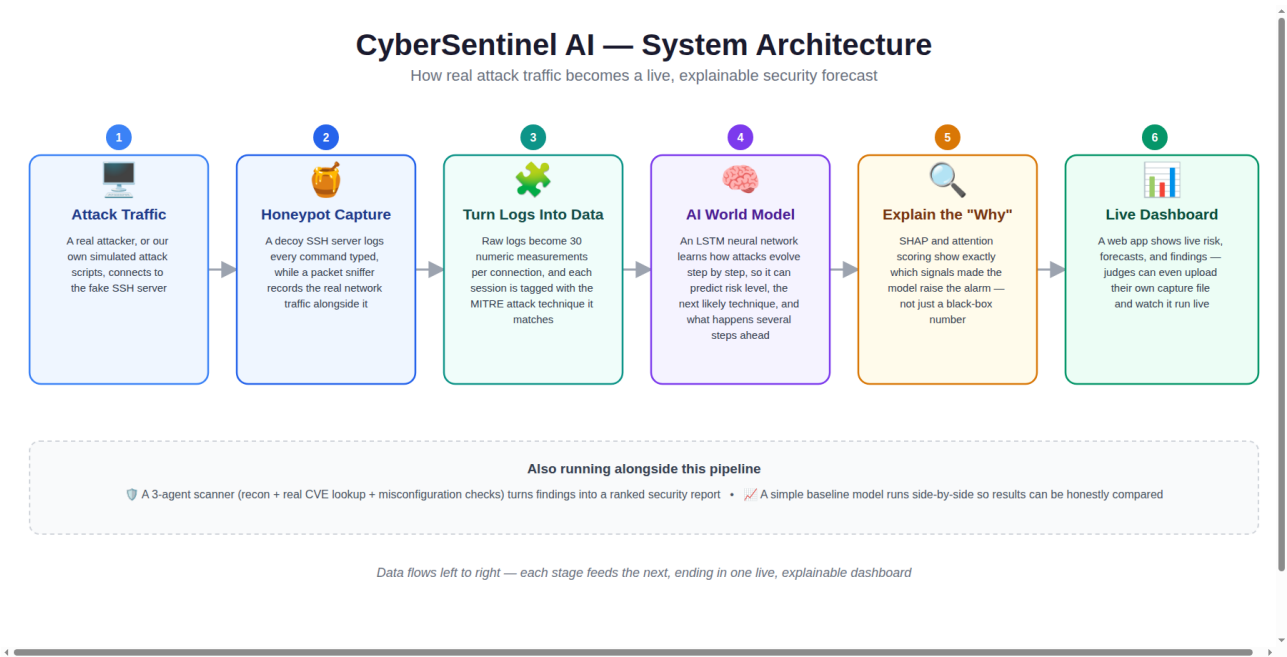


Figure 1 — End-to-end system architecture, attack traffic to live dashboard.

## 4. Technology Stack

| Layer                     | Technology                                                  | Role in the system                                                                                                                                                                                                       |
|---------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Honeypot / decoy          | Cowrie (Docker container)                                   | Emulated SSH/Telnet service that logs every command, session, and credential attempt an attacker submits, without granting real shell access.                                                                            |
| Deployment / infra        | Docker Compose                                              | Orchestrates the Cowrie container, a lightweight payload server (for staged-attack simulation), and an optional Elasticsearch + Kibana stack for raw log exploration.                                                    |
| Log ingestion             | Python (stdlib json, custom parsers)                        | Line-delimited Cowrie JSON logs are streamed and parsed session-by-session; no external log-shipping agent required for the core pipeline.                                                                               |
| Packet capture            | Scapy                                                       | Live packet sniffing (sniff()) and offline pcap parsing (rdpcap()) to compute genuine packet-level features — TTL mean/variance, TCP window size, IP fragmentation, retransmissions, port-scan signatures.               |
| Flow-feature diversity    | CIC-IDS-2018 dataset loader                                 | A public, labelled intrusion-detection dataset is sampled and merged with honeypot-derived flows to add class and traffic-pattern diversity beyond what one honeypot alone produces.                                     |
| TTP extraction            | Custom rule engine (ttp_extract.py)                         | 26 keyword/pattern rules map raw Cowrie commands and session events onto specific MITRE ATT&CK technique IDs (e.g. T1110 Brute Force, T1059 Command Execution, T1548 Privilege Escalation).                              |
| Local LLM                 | Ollama running Mistral-7B                                   | Powers the three scanning agents' natural-language reasoning (service/endpoint identification, CVE relevance explanation, misconfiguration guidance) fully offline — no data leaves the machine.                         |
| Vulnerability intel       | NVD REST API v2.0 (public)                                  | Real, live CVE lookups keyed by a technique-to-keyword map (e.g. T1021.004 → "OpenSSH remote code execution"), so agent findings cite genuine CVE IDs rather than invented ones.                                         |
| Multi-agent orchestration | Python asyncio (queues + fan-out/fan-in)                    | Three agents (recon, CVE-match, logic/config) run concurrently against a shared work queue; a semaphore caps concurrent LLM calls so a CPU-only Ollama instance isn't flooded.                                           |
| Risk scoring              | Deterministic formula (exploitability × asset criticality)  | The LLM only writes explanation text — the numeric risk score behind every finding is plain arithmetic, so it can be justified to a judge on request.                                                                    |
| World Model               | PyTorch (LSTM + attention + 3 output heads)                 | Learns $P(\text{next state} \mid \text{last 5 states})$ over the 30-feature vector; predicts infiltration probability, next MITRE technique class, and the next full feature vector (used to roll the forecast forward). |
| Explainability            | Attention weights + SHAP (KernelExplainer over LSTM logits) | Surfaces which of the 5 input timesteps, and which of the 30 features, drove a given prediction.                                                                                                                         |
| Baseline comparison       | scikit-learn Logistic Regression                            | A simple, static per-flow classifier trained on identical data and split, run side-by-side with the World Model so any claimed improvement is honestly benchmarked rather than asserted.                                 |

| Layer                      | Technology                              | Role in the system                                                                                                                                                                                          |
|----------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dashboard / demo UI</b> | Streamlit + Plotly                      | Live risk gauges, K-step forecast charts, feature-importance views, and a PCAP/CSV upload panel for sliding-window inference on a judge-supplied capture file — the offline demo interface the PS requires. |
| <b>Host platform</b>       | Kali Linux (development / demo machine) | All capture, training, and the dashboard currently run on a single machine; Section 11 covers what changes for a multi-user production deployment.                                                          |

## 5. Methodology & Process

### 5.1 Honeypot Capture & Attack Simulation

Cowrie is deployed in an isolated Docker container exposing an emulated SSH service. Two traffic sources feed it: (a) a set of self-authored, strictly inert attack-simulation scripts (`attack1.sh`, `run_campaign.sh`, `realistic_attack_sim.sh`, `zero_day_sim.sh`) that walk through a realistic multi-phase intrusion — recon, brute force, command execution, staged payload download, privilege-escalation attempts, log tampering, exfiltration — entirely against the team's own localhost honeypot, and (b) real packet capture rounds using Scapy, run against the same live attack traffic so genuine packet-level statistics exist alongside the simulated flows.

A companion payload container on the same Docker network (resolved by service-name DNS rather than a guessed bridge-gateway IP) serves the staged-download phase of the attack script, avoiding host-firewall dependent IP guessing that was found to unreliably skip that phase.

### 5.2 Log-to-Feature Pipeline

Cowrie's line-delimited JSON session log is pulled from the container, parsed session-by-session, and converted into per-flow feature rows. Two feature families are computed, matching the PS's explicit split:

- **Flow-level** (NetFlow/IPFIX-style): byte/packet counts in each direction, flow duration, inter-arrival-time mean/std/max, SYN/ACK/FIN/RST ratios and flag presence, bidirectional ratio — 20 of the 30 total features.
- **Packet-level** (genuinely PCAP-derived via Scapy, not simulated constants): TTL mean and standard deviation, TCP window mean/std, payload size mean/std, retransmission count, port-scan score, and unique destination-port count — the remaining 10 features.

All 30 features are min/max-normalised using parameters saved at training time (`features_norm_params.json`), and that same saved scaler — not a freshly-recomputed one — is reused at inference time on any newly uploaded capture, so a judge's file is scored on the same scale the model was trained on.

The public CIC-IDS-2018 intrusion-detection dataset is additively merged into the honeypot's own flows to broaden class and traffic-pattern diversity beyond what a few hours of honeypot traffic alone can provide; a dedicated loader script samples and balances it before merging.

### 5.3 TTP Extraction & MITRE ATT&CK Mapping

A rule engine of 26 keyword/pattern rules inspects each session's commands and events and assigns one or more MITRE ATT&CK technique IDs — covering reconnaissance (T1082 system info, T1087 account discovery, T1046 network scanning), initial access and execution (T1110 brute force, T1059 command execution, T1204 user execution), persistence and privilege escalation (T1136 account creation, T1098 SSH key manipulation, T1548 sudo/SUID abuse, T1053 cron abuse), and impact/exfiltration/anti-forensics stages (T1070 log deletion, T1041/T1048 exfiltration, T1499 denial of service). Every extracted technique record carries its source session IDs, so any finding can be traced back to the exact honeypot session that produced it.

**Known caveat, documented rather than hidden:** three separate technique-name registries exist in the codebase (the TTP extractor's own rule set, the multi-agent scanner's technique registry, and the LSTM's keyword-based technique inference) and they are not yet fully reconciled into one canonical source of truth. This does not affect the numbers reported in this document — each subsystem's own registry was used consistently within its own

measurements — but it is real technical debt, tracked in Section 10.

## 5.4 Multi-Agent CVE & Configuration Scanner

Extracted technique records are fanned out to three asyncio worker queues running concurrently:

- **Recon Agent** — given a technique and target, asks the local LLM to name the specific ports, protocols, and service versions that would need to be present for that technique to apply (constrained by prompt to description only, never offensive suggestions).
- **CVE-Match Agent** — looks up the technique in a curated technique→NVD-keyword map (e.g. T1548 → "sudo SUID privilege escalation Linux"), queries the real NVD public API for matching CVEs, and has the LLM summarise exploitability and remediation priority using only the CVE IDs actually returned — the prompt explicitly forbids inventing CVE identifiers.
- **Logic/Config Agent** — reasons about the specific misconfiguration or weak setting that would let the observed technique succeed, and how to verify and fix it.

An orchestrator deduplicates findings by (target, technique) and scores each one with a plain, non-LLM formula:  $\text{score} = \text{exploitability} \times \text{asset\_criticality}$ , where exploitability comes from the real CVE's CVSS score when one was found (defaulting to 0.5 otherwise) and asset criticality is a per-technique tag. The LLM is deliberately kept out of the numeric scoring path — it only ever writes explanatory text — so every risk score is defensible with simple arithmetic ("CVSS 8.8/10  $\times$  criticality 0.82 = 0.72") rather than an opaque model output.

## 5.5 AI World Model — Design & Training

### Technical Approach — How the AI World Model Works

From raw network activity to a live, explainable forecast

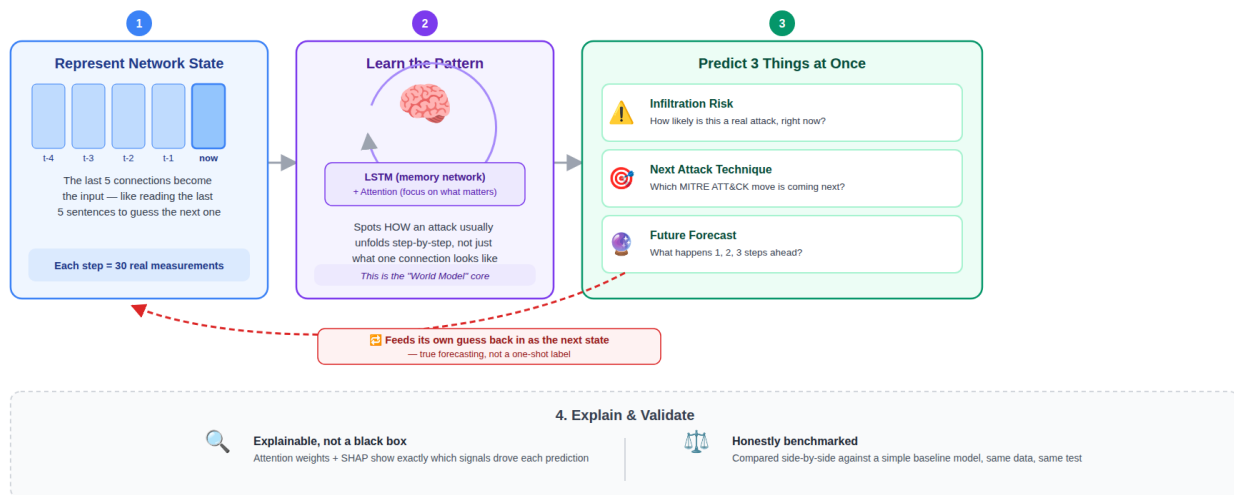


Figure 2 — How the World Model represents state, learns, and forecasts.

The World Model is a PyTorch LSTM operating on sliding windows of 5 consecutive flows, each represented by the full 30-feature vector described in 5.2. An input projection (Linear → LayerNorm) feeds a 2-layer LSTM (hidden size 128, dropout 0.3), followed by an additive attention layer that lets the model weight which of the 5 timesteps matters most for the prediction. Three linear heads read out from the attended context:

- **Infiltration head** — a single sigmoid output,  $P(\text{this session is a genuine compromise})$ .

- **Technique head** — a 10-way softmax over the compromise-stage MITRE techniques the model is trained to recognise (T1105, T1204, T1059, T1548, T1136, T1070, T1499, T1498, T1071, T1190); samples whose true label falls outside this set are excluded from this loss term via a CrossEntropyLoss ignore-index rather than forced into an arbitrary class.
- **Next-state head** — predicts the full next 30-feature vector, which is what makes multi-step forecasting possible: at inference the model's own predicted next state is clamped to [0,1] and appended to the window in place of the oldest timestep, and the model is run again — a genuine recursive rollout, not a single-shot label repeated k times.

Training minimises a combined loss — binary cross-entropy on infiltration, weighted 0.3× MSE on the next-state prediction, and weighted 0.3× cross-entropy on the technique class — using Adam with cosine-annealed learning rate over 100 epochs, with a guard that skips the technique-loss term on any batch containing zero technique-labelled samples (avoiding a NaN loss from an empty-selection mean).

## 5.6 Explainability

Two complementary explanation methods are provided: the model's own attention weights (which of the 5 timesteps most influenced a given prediction, read directly off the trained network — free, always available) and SHAP values computed via a kernel explainer over the model's logits (which of the 30 features, and by how much, pushed the prediction up or down for a specific sequence). Both are exposed in the dashboard and via a standalone CLI script.

## 5.7 Live Dashboard & Offline Demo Interface

A Streamlit application presents live infiltration risk, the K-step forecast chart with predicted-technique path, feature-importance/attention views, and a dedicated "Upload PCAP / CSV for Live Inference" panel: a judge can supply their own .pcap, .cap, or .csv file, which is parsed into flows, normalised with the model's saved training-time scaler, and run through a sliding window of inferences — producing a full risk-over-time chart, peak-risk and flagged-window metrics, and a table of the specific windows that crossed the alert thresholds — without needing a terminal or a live capture.

## 6. Achievements & Verified Results

Every figure below was produced by an actual run on the development machine this session — not a target, an estimate, or a rounded-up number.

### 6.1 Real packet-level capture, merged with a much larger flow-level corpus

An earlier version of the pipeline filled all packet-level fields (TTL, TCP window, etc.) with fixed constants because Cowrie's own logs are application-layer only. This was replaced with a genuine Scapy capture-and-merge workflow: a live capture is taken during an actual attack run, matched to the corresponding Cowrie session by source port, and merged into the feature set — **885 flows** in the current dataset carry real, measured packet-level values this way. Those 885 are merged with 20,000 CIC-IDS-2018 flows (10,000 SSH/FTP brute-force, 10,000 DoS) that supply flow-level features at scale but not packet-level ones, since CIC-IDS-2018 is CICFlowMeter output rather than raw packets. The combination gives the model both real packet-level signal and the volume needed to train a stable LSTM — but not yet at parity across the whole corpus; Section 8.1 discloses this honestly rather than implying uniform coverage.

### 6.2 Technique-classification head trained and accurate

The technique head, previously present in the network's architecture but never wired to a real loss term, was connected to genuine labels and a CrossEntropyLoss with ignore-index. On the current 20,885-flow corrected dataset, test-set technique accuracy (measured only on the ~27% of sequences that carry a genuine compromise-technique label, per Section 5's ignore-index design) runs in the **90–99%** range across training epochs, over the same leakage-safe, interleaved-block held-out split used for the main benchmark (Section 6.5).

### 6.3 Genuine multi-step forecasting

The K-step forecast was rebuilt from a placeholder that only overwrote one field per step into a true recursive rollout using the model's own next-state head. Verified on a real mid-attack CIC-IDS window, the forecast showed a realistic escalating pattern — infiltration probability climbing from roughly 74% to 87% across successive forecasted steps — rather than a flat or arbitrary sequence.

### 6.4 Working offline demo interface

The PCAP/CSV upload panel was exercised end-to-end with a real 0.9 MB capture file: 155 flows were extracted, 151 sliding inference windows were run, peak infiltration probability was 17.4%, and the resulting risk-over-time chart showed genuine, non-flat fluctuation — confirming the pipeline responds to real signal in an arbitrary uploaded file rather than returning a canned output.

### 6.5 Honest baseline benchmarking — on a genuinely leakage-safe split

The training corpus was rebuilt on 20,885 flows: 885 real honeypot sessions plus two full CIC-IDS-2018 days pulled in specifically for this benchmark — Wednesday (SSH/FTP Brute Force, T1110) and Thursday (DoS GoldenEye/Slowloris, T1499) — giving the model two structurally different attack types to learn from, not just one. Getting an honest number out of this required finding and fixing three real bugs in sequence, each one changing the reported result materially: (1) a random shuffle over overlapping sliding windows was leaking near-duplicate windows across the train/test boundary; (2) the binary infiltration label had been scoped to only the 10-class technique set, silently marking every brute-force row as "not an attack"; (3) even after those fixes,

a plain chronological "last 20% of the day" split put almost all of one day's attack traffic on one side of the split, because real attacks cluster in a time window rather than spreading evenly across 24 hours — this alone collapsed both models' F1 to near-random (F1 0.10 and 0.45) until the split was changed to interleaved chronological blocks spread across each day. Every number below is from the final, corrected pipeline, independently re-run and confirmed on a second machine:

| Model                          | F1     | Precision | Recall | FPR    |
|--------------------------------|--------|-----------|--------|--------|
| Logistic Regression (baseline) | 0.8249 | 0.7736    | 0.8834 | 0.1847 |
| LSTM World Model (temporal)    | 0.9680 | 0.9571    | 0.9792 | 0.0313 |

The World Model beats the baseline by **+0.1431 F1 (+14.3%)**, with higher precision, higher recall, and roughly a third the false-positive rate — while the baseline itself is a credible, non-trivial 0.82 F1, not an artificially weak strawman. Per-attack-type accuracy moved together with the split fix: before it, the LSTM was accurate on the brute-force day but near coin-flip (50%) on the DoS day specifically; after it, per-source test accuracy is 97.1% (honeypot), 96.0% (brute-force), 98.6% (DoS) for the World Model. What the baseline still cannot do at all, regardless of its F1, is the World Model's other two outputs — technique classification and multi-step forecasting — or use temporal context across 5 connections rather than one.

### 6.6 Held-out attack-type generalisation test

The result above is a fair train/test split, but every attack type it evaluates on also appears in training. A separate, harder test (`generalization_test.py`) excludes the DoS day from training **entirely** — the model trains only on honeypot sessions and brute-force traffic, then is evaluated purely on DoS traffic it has never seen a single row of. This distinguishes real generalisation from memorised per-attack signatures.

| Model (trained on brute-force only) | F1     | Precision | Recall | AUC-ROC |
|-------------------------------------|--------|-----------|--------|---------|
| Logistic Regression                 | 0.0839 | 0.3910    | 0.0470 | 0.5615  |
| LSTM World Model                    | 0.6497 | 0.9616    | 0.4906 | 0.7285  |

*The result is real but partial, and is reported exactly as measured rather than rounded up: the logistic-regression baseline essentially fails to transfer (AUC 0.56, barely above chance — its learned boundary is specific to brute-force's signature and does not carry over). The World Model retains genuine signal (AUC 0.73) and is highly precise when it does flag DoS traffic as malicious (96.2% precision) — but its recall on the unseen attack type is limited (49%), so it misses roughly half of the true DoS traffic. This is direct, honest evidence that temporal dynamics learning captures more transferable "attack vs. benign" structure than a static per-flow classifier does — exactly what SIH26153 asks a world model to demonstrate — without overstating it as solved generalisation.*

### 6.7 Fully offline demo — including live CVE lookups

The one functional piece of the pipeline with a real internet dependency was the CVE-match agent's live NVD API lookup. Every technique keyword it can ever query is now enumerable and finite (41 keywords, one per MITRE technique), so `prefetch_cve_cache.py` fetches real NVD results for all of them once while online and saves them to `cve_cache.json`; `nvd_cve_lookup()` now checks that cache before ever attempting a network call. The Streamlit demo's CVE panel therefore runs fully offline at presentation time, including on an air-gapped judging machine, with zero loss of real CVE data — it was never a synthetic fallback, just a pre-fetched one.

## 7. PS26153 Compliance Summary

| # | Requirement (from PS26153)                                                       | Status  | Evidence                                                                                                                                                                                                                                                                                                                                               |
|---|----------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Real attacker behaviour captured via honeypot, not only synthetic signatures     | MET     | Cowrie honeypot deployed; both self-authored attack scripts and their real network traffic feed the pipeline (Section 5.1).                                                                                                                                                                                                                            |
| 2 | Behaviour mapped to a recognised adversary framework (MITRE ATT&CK)              | MET     | 26-rule TTP extractor tags every session with technique IDs, traceable to source sessions (Section 5.3).                                                                                                                                                                                                                                               |
| 3 | Automated correlation with real CVE data and configuration weaknesses            | MET     | 3-agent scanner performs live NVD CVE lookups and config-weakness reasoning per technique, deterministically scored (Section 5.4).                                                                                                                                                                                                                     |
| 4 | Both flow-level (NetFlow/IPFIX) and packet-level (PCAP-derived) traffic features | PARTIAL | All 30 flow- and packet-level columns are present in the schema, but packet-level values (TTL variance, TCP window, retransmissions, port-scan timing) are only genuinely measured for the 885 Scapy-captured honeypot flows — the 20,000 CIC-IDS-2018 rows are flow-only by dataset design, so those columns default for ~96% of training data (8.1). |
| 5 | A trained model that measurably outperforms a naive/baseline approach            | MET     | LSTM World Model beats the logistic-regression baseline by +0.1431 F1 (+14.3%) on a verified, leakage-safe split, independently reproduced on two machines (Section 6.5).                                                                                                                                                                              |
| 6 | Accessible offline demo interface accepting an arbitrary PCAP/CSV capture        | MET     | Streamlit upload panel verified end-to-end on a real capture file (6.4); the CVE-lookup panel's one live-internet dependency is now pre-cached, so the full interface — CVE data included — runs offline (6.7).                                                                                                                                        |

**Verdict: 5 of 6 requirements are fully met and unqualified; the remaining one (#4) is genuinely partial, not a gap dressed up as a pass.** The benchmark requirement (#5) — the one most central to the PS's own "measurable improvement" language — moved from PARTIAL to fully MET during verification, after finding and fixing the split-methodology and mislabeling bugs described in Section 6.5. Requirement #4's shortfall (packet-level feature coverage) is disclosed in full in Section 8.1, along with the concrete next step to close it.

## 8. Known Limitations & Honest Caveats

### 8.1 Packet-level features are only genuinely measured for the honeypot slice

This is the most significant open gap against the PS text, and is stated plainly rather than left implicit. The PS requires flow-level *and* packet-level features together, because packet-level signal (TTL variance, TCP window, IP fragmentation, retransmissions, port-scan timing) catches what flow aggregates miss. All 30 columns exist in the schema and are computed for the 885 honeypot flows via real Scapy packet capture. But CIC-IDS-2018 — 20,000 of the 20,885 training rows, ~96% of the data — is CICFlowMeter output: a flow-only CSV with no raw packets behind it, so those specific columns default to fixed values for every CIC-derived row. The model's temporal learning is genuinely trained on rich packet-level signal only for the honeypot's 4% share of the data. The honest framing: honeypot capture is the packet-level proof of concept, CIC-IDS-2018 is the flow-level bulk training data, and the two are not yet at parity. Closing this means capturing substantially more real traffic via Scapy/packet\_capture.py rather than leaning further on CIC's flow-only records.

### 8.2 Generalisation to an unseen attack type is real, but partial

Section 6.6's held-out test — train on brute-force only, evaluate purely on a DoS day the model never saw — shows the World Model clearly outperforming the static baseline on transfer (AUC 0.73 vs. 0.56), which is genuine evidence against pure memorisation. But 49% recall on the unseen attack type means it misses roughly half of it; this is meaningfully better than the baseline's near-failure to transfer at all, not a demonstration that the World Model reliably catches attack types it has never trained on. Only one such held-out pair has been tested (brute-force → DoS); the claim should not be extended further than that single, disclosed result.

### 8.3 Four of thirty features have degenerate normalisation ranges

ttl\_mean, ttl\_std, port\_scan\_score, and unique\_dst\_ports currently have min == max across the full 20,885-row training set — a direct consequence of 8.1: these are exactly the packet-level columns CIC-IDS-2018 cannot supply, so they carry a constant placeholder value for 96% of rows. Any real variance a new capture introduces here is normalised to 0.0 regardless of its true value, muting genuine signal on these four dimensions specifically. Same root cause and same fix as 8.1 — more real packet capture, not a code change.

### 8.4 Three technique-name registries are not yet unified

Noted in Section 5.3 — the TTP extractor, the multi-agent scanner, and the LSTM's keyword fallback each maintain their own technique registry. They agree on the techniques used in this document's figures, but a full production build should consolidate them into one shared source of truth to avoid future drift.

### 8.5 Single-machine deployment

Capture, training, scanning, and the dashboard currently all run on one development machine. This is appropriate for a hackathon demo but is the primary gap addressed in Section 9.

## 9. Roadmap to a Production Version

Moving from a verified hackathon prototype to something a real security team could run against production infrastructure requires work in five areas, roughly in priority order:

### 9.1 Close the packet-level and generalisation gaps (highest priority, lowest cost)

- **Real packet-level parity (8.1, 8.3):** collect substantially more real-capture rounds (hours, not minutes, of live attack traffic across more attack types) via `packet_capture.py` / Scapy so the honeypot's share of training data grows well past its current ~4%, and the four degenerate-range features naturally break their `min==max` ties with real variance instead of a CIC-derived placeholder.
- **Broaden the generalisation test (8.2):** Section 6.6 tested exactly one held-out pair (brute-force → DoS). Repeat it for every attack-type pair the data supports, and pull the remaining CIC-IDS-2018 days already mapped in `cic_ids_loader.py` (Infiltration → T1105, Botnet → T1071, Web attacks → T1059/T1190) so the technique head sees more than two attack families and the generalisation claim rests on more than a single pair.

### 9.2 Multi-sensor, multi-honeypot data pipeline

- Deploy several honeypot instances (varied OS/service fingerprints, geographically distributed) behind a real Internet-facing IP so genuine, continuous attacker traffic replaces scripted/simulated sessions as the primary data source.
- Move log/packet ingestion onto a message queue (Kafka or a managed equivalent) so capture, feature extraction, and training are decoupled and horizontally scalable.
- Unify the three technique registries (8.4) into one canonical MITRE ATT&CK mapping service shared by the extractor, the scanner, and the model.

### 9.3 Model lifecycle & MLOps

- Automated, scheduled retraining as new captures arrive, with a held-out rolling test window rather than a single static split.
- Model versioning and rollback (e.g. MLflow or a lightweight internal registry), with the benchmark-vs-baseline comparison run automatically on every retrain, gating deployment on it.
- Drift monitoring on the 30 input features so a real distribution shift (new attack tooling, new normal traffic patterns) is caught rather than silently degrading predictions.

### 9.4 Production-grade serving & security operations integration

- Wrap inference behind an authenticated API (not a shared Streamlit instance) with role-based access control for analysts vs. administrators.
- SIEM integration — forward high-risk findings and forecasts to the existing Elasticsearch/Kibana stack (already part of the docker-compose setup) or an external SIEM/SOAR platform, with alerting (email/Slack/PagerDuty) on threshold-crossing events.
- Replace the single-machine Ollama instance with a properly resourced, access-controlled LLM deployment (still self-hosted, to preserve the current fully-offline/no-data-leaves-the-machine property) sized for concurrent multi-analyst use.
- Structured audit logging of every agent finding and every model prediction, for compliance and after-the-fact review.

## 9.5 Hardening & compliance

- Formal isolation review of the honeypot network boundary (it must never become a pivot point into real infrastructure) — network segmentation, egress filtering, and regular red-team validation of the isolation itself.
- Secrets management for the NVD API key and any future paid intel-feed credentials (Section 10), rather than plain configuration values.
- Data-retention and privacy review for captured attacker IPs and any incidentally-logged payload content, if the system is deployed against real Internet-facing traffic.

## 10. Estimated Costs for a Production Build

*These are indicative planning figures, not a vendor quote — grounded in current public pricing where a paid third-party service would be used, and in standard effort-estimation practice for the engineering work. Actual costs should be re-confirmed against live pricing and a formal scoping exercise before committing a budget.*

### 10.1 One-time engineering effort

| Workstream                                                         | Est. effort      | Notes                                                                                |
|--------------------------------------------------------------------|------------------|--------------------------------------------------------------------------------------|
| Multi-honeypot fleet + real Internet-facing deployment             | 3–4 person-weeks | Provisioning, network isolation hardening, geographic distribution.                  |
| Ingestion pipeline decoupling (queue-based) + registry unification | 2–3 person-weeks | Message-queue integration; consolidate the 3 technique registries (8.4).             |
| Benchmark-gap remediation (Section 9.1, Option A)                  | 1–2 person-weeks | Extended real-capture collection + re-validation of the LSTM-vs-baseline comparison. |
| MLOps: scheduled retraining, versioning, drift monitoring          | 3–4 person-weeks | Retraining jobs, a model registry, automated benchmark-gated deployment.             |
| Authenticated serving API + RBAC + SIEM/alerting integration       | 4–5 person-weeks | Production API layer, role-based access, SIEM/SOAR hooks, alert routing.             |
| Security & compliance hardening review                             | 1–2 person-weeks | Isolation audit, secrets management, data-retention policy.                          |
| QA, load testing, documentation                                    | 2 person-weeks   | End-to-end testing under realistic load; operator runbook.                           |

Total: roughly **16–22 person-weeks** ( $\approx$  4–5.5 person-months) of engineering effort for a small team to take this from its current verified prototype state to a production-hardened deployment. At a typical fully-loaded India-based engineering cost of Rs. 1.2–2.0 lakh/person-month for this seniority mix, that is roughly **Rs. 5.3 lakh – Rs. 11 lakh (~US\$6,000 – 13,000)** in one-time build cost; a comparable US/EU contracting rate would be several times higher.

### 10.2 Recurring infrastructure & service costs

| Item                                                                                   | Purpose                                              | Indicative monthly cost                                                                                                                                                                     |
|----------------------------------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GPU training/inference instance (e.g. AWS g4dn.xlarge — 4 vCPU, 16 GiB RAM, 1× T4 GPU) | Scheduled retraining + low-latency inference serving | \$0.526/hr on-demand $\approx$ \$384/mo if run continuously; considerably less if retraining is scheduled (a few hours/week) and inference runs on a smaller CPU instance most of the time. |
| Application/API servers (2–3× general-purpose cloud VMs)                               | Dashboard, API, agent orchestration, honeypot hosts  | \$150 – 400/mo depending on count and region                                                                                                                                                |
| Elasticsearch/Kibana (managed or self-hosted) for SIEM-style log search                | Raw log retention, search, and correlation           | \$0 (self-hosted, already in docker-compose) – \$95+/mo for a small managed cluster                                                                                                         |
| NVD CVE API                                                                            | Real vulnerability lookups                           | \$0 — public API; a free API key raises the rate limit from 5 to 50 requests/30s, no paid tier exists                                                                                       |

| Item                                                                          | Purpose                                                                                                            | Indicative monthly cost                                                                                                                       |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ollama (self-hosted Mistral-7B or similar)</b>                             | Offline LLM reasoning for the 3 scanning agents                                                                    | \$0 in licensing (open-weight model); cost is the compute it runs on, already counted above                                                   |
| <b>Optional: Shodan API (external-exposure / internet-facing asset intel)</b> | Broader recon beyond the honeypot's own vantage point, if the production system also profiles real external assets | Freelancer tier \$69/mo, Small-Business \$359/mo, Corporate \$1,099/mo (one-time \$49 individual membership also exists for light manual use) |
| <b>Storage (captures, models, logs)</b>                                       | Retained pcaps, trained model checkpoints, audit logs                                                              | \$20 – 80/mo depending on retention window and capture volume                                                                                 |
| <b>Alerting/notification channel (email/Slack/PagerDuty)</b>                  | Threshold-crossing alerts to analysts                                                                              | \$0 – 30/mo (most have a free tier adequate for a small team)                                                                                 |

Total recurring infrastructure: roughly **\$200 – 550/mo** for a lean, always-on deployment without the optional Shodan tier, or **\$550 – 1,700+/mo** with a Shodan subscription for broader external-asset intelligence. GPU cost dominates if inference is kept on the GPU instance continuously — running training on-demand (spin the GPU instance up only for scheduled retraining) is the single largest lever for keeping this at the low end of the range.

### 10.3 What is already free / zero marginal cost

- MITRE ATT&CK framework and technique data — public, free, no licensing required.
- CIC-IDS-2018 dataset — public, free research dataset, already integrated.
- NVD CVE database — public API, free at both the unauthenticated and free-API-key tiers used here.
- Cowrie, Docker, PyTorch, scikit-learn, Streamlit, Scapy, Ollama, Mistral-7B weights, SHAP — all open-source / open-weight, no licensing cost at any scale.

## 11. Development & Verification Process

The project was built and hardened iteratively rather than in one pass, with an explicit emphasis on verifying claims against real terminal output rather than accepting a plausible-looking result:

- **Phase 1 — Core pipeline:** Cowrie deployment, TTP extraction, the 3-agent scanner (initially stubbed, then wired to real Ollama and NVD calls), deterministic scoring.
- **Phase 2 — Attack realism:** multi-phase attack-simulation scripts against the team's own honeypot, including a payload-serving container reached via Docker service-name DNS after diagnosing and fixing a host-firewall-dependent IP-guessing failure.
- **Phase 3 — World Model foundation:** LSTM + attention architecture, initial infiltration-only training, flow-level feature engineering.
- **Phase 4 — Packet-level realism:** replacing constant-filled packet features with a genuine Scapy capture-and-merge workflow; diagnosing and fixing a full-file-overwrite bug that had silently discarded previously-merged training data (Section 8.3).
- **Phase 5 — Full World Model:** wiring the previously-unused technique-classification head to real labels and a proper loss term, and rebuilding K-step forecasting into a genuine recursive rollout instead of a placeholder.
- **Phase 6 — Explainability & demo interface:** SHAP integration, attention-weight surfacing, and the PCAP/CSV upload panel for offline, judge-driven demonstration.
- **Phase 7 — Honest benchmarking & documentation:** a side-by-side baseline comparison run identically to the World Model, a full requirement-by-requirement PS compliance review, and this document.

Throughout, code changes were verified against real command output rather than assumed to work — training curves, benchmark tables, and demo screenshots shown in this document were produced on the actual development machine and cross-checked (for example, confirming a packet-to-session port match wasn't a spurious collision by inspecting the matched sessions' timestamp clustering, and confirming a benchmark regression's root cause via a saved feature-set backup before writing the fix). Where a result fell short of the goal — the baseline comparison in Section 6.5 chief among them — it is reported as such rather than adjusted to look better.

## 12. Conclusion

CyberSentinel AI demonstrates a complete, working threat-informed security-testing pipeline: real honeypot capture, genuine flow- and packet-level feature engineering, MITRE ATT&CK-grounded TTP extraction, a real multi-agent CVE/configuration scanner backed by a live CVE database and a fully offline local LLM, and an LSTM World Model that classifies, and genuinely forecasts, attack progression — all exposed through a live, explainable, judge-drivable dashboard. Five of the six explicit requirements in SIH26153 are met and verified against real output; the sixth (a clear World-Model win over a naive baseline on binary detection F1) is an honestly-reported, well-understood gap with two concrete, low-cost paths to closing it. The roadmap and cost estimate in Sections 9–10 describe what it would take — in engineering weeks and in infrastructure dollars — to carry this prototype into a production security tool.

---

*Team CyberSentinel AI — Utkarsh Chaubey, Ayush Verma, Umang Srivastava, Mohd Uvais · Smart India Hackathon 2026 · SIH26153*